Searching

1.

You are given an array containing N distinct integers in a wave-like sequence. Meaning, the numbers in the beginning are in ascending order, and after a specific position, they are in descending order. For example: [1, 3, 4, 5, 9, 6, 2, -1]
You have to find the maximum number of this sequence. Can you devise an efficient algorithm such that the time complexity will be less than $O(N)$?

a) **Present** your solution idea as a pseudocode/ python code/ flowchart/ step-by-step instructions/ logical explanation in one-two paragraphs.
b) **Write** the time complexity of your algorithm.

2.

You are given an array containing N distinct integers in a wave-like sequence. Meaning, the numbers in the beginning are in descending order, and after a specific position, they are in ascending order. For example: [9, 7, 6, 4, 2, 1, 3, 5, 8]
You have to find the minimum number of this sequence. Can you devise an efficient algorithm such that the time complexity will be less than $O(N)$?

c) **Present** your solution idea as a pseudocode/ python code/ flowchart/ step-by-step instructions/ logical explanation in one-two paragraphs.
d) **Write** the time complexity of your algorithm.

3.

Consider an array containing **N** unique values where for some index **i**, the values are in increasing order from index **0** to **(i-1)**, and then again from **i** to **(N-1)**. An example array is given below.
Here **i=3**, it means the values are in increasing order from index 0 to 2, and then again from 3 to 7.

index	0	1	2	3	4	5	6	7
value	9	12	15	2	4	5	7	8

Given the value of **i**, propose an algorithm to search a *key_value* in the array. Complexity of your algorithm must be less than **O(N)**.

a) Present your solution idea as a code/ pseudocode/ flowchart/ step-by-step instructions/ logical explanations in short.
b) Write the time complexity of your presented solution.
c) Show a simulation of the Merge Sort algorithm to organize the whole array in increasing order.

4.

Your friend gave you an integer (`key`) and asked you to find its integer square root without using built-in functions like `sqrt()` or exponentiation. For example, if the input is 25, the output should be 5 because $5^2 = 25$. If the square root is not an integer, return the floor value of the actual square root. For example, for an input of 10, the output should be 3.

Your friend's initial approach is to use linear search (time complexity $O(n)$) as shown below.

```
def LinearSearchToFindSquareRoot(key):
     result = -1
     for i in range(1, key, 1):
         if i * i <= key:
             result = i
     return result
```

Now to find the square root he is working on a possible solution **P,** which will result in time complexity $O(\sqrt{n})$. As an algorithm student, you propose another algorithm **R** within time complexity $O(log_2 N)$, which is much faster than $O(\sqrt{n})$.

i. Present the **solution your friend** was trying to achieve **(Algorithm P)** with pseudocode/programmable code/step-by-step logical instructions.
ii. Present your proposed solution **(Algorithm R)** with pseudocode/programmable code/step-by-step logical instructions.

5.

Given an array of **N** elements, you are required to find the **peak element** in it. An element is considered to be a peak if its value is greater than the value(s) of its adjacent element(s) i.e. immediate left and immediate right. If there is only one adjacent element, we take only that in consideration. And the array will contain **exactly one** peak element.

Now answer the following questions.

i. **Determine** whether each of the following arrays matches the above stem-mentioned conditions:
   1. [1, 5, 4, 3, 2]
   2. [1, 3, 2, 5, 4]
   3. [5, 2, 1, 3, 4]
   4. [1, 2, 3, 4, 5]

   [N.B.: For each array, it is enough to write only True/False in the answer script.]

ii. **Present** an algorithm with a time complexity lower than **O(N)** to solve the stem-mentioned problem using pseudocode/ programmable code/ step-by-step logical instructions.
iii. If the array contains **more than one** peak element, will the presented algorithm in question (ii) still be able to find any peak element? **Explain** briefly.

6.

You are given an array containing N distinct integers in ascending order. You have to find out the first missing number from the increasing sequence.
For example: [3, 4, 5, 6, 9, 10]. The first missing number in this sequence is 7.
[Explanation : If A_i and A_{i+1} are two consecutive integers from the list and their difference $(A_{i+1} - A_i)$ is more than 1, then we say A_i+1 is a missing number.]

Now that you have understood the problem completely, you are trying to solve it efficiently. Your friend, Jack, said that by iterating through all the numbers from the beginning, you can find the first missing number in linear time $[O(N)]$. But, you need a more efficient one. Now, **answer** the questions below.

a) If $N = 9$ and the maximum number (obviously the last-indexed one) in the array is 15, which one among 6, 8, 10 might be the first number (obviously the minimum one) in the array?
b) Explain why you discarded the other two numbers in question no. (a).

c) Assume $N = 7$ and the minimum and maximum numbers are 2 and 12 respectively. You found the middle-index value to be 5. On which side (left/right) of 5, do you expect to find the first missing number? Only mention 'left' or 'right'.
d) Explain why you discarded the other half of the array in question no. (c).
e) Assume $N = 7$ and the minimum and maximum numbers are 8 and 19 respectively. The missing elements are 11, 13, 14, 17, 18. What will be the value of the middle-index element? Determine the absolute difference between the middle-index value and the maximum number.

7.

For different tasks, we often need to search for things. And most of the time, we sort the dataset before performing search operations. Once, you found the following list of numbers.

[2, 5, 1.2, 6.7, 1.7, 9.3, 2.2, 7.7, 0, –4, –5.1, 2, 5, 5.2]

a. To search an element in an unsorted array, we need O(N) time using linear search. Binary search works in O(logN) time but the array needs to be sorted beforehand which takes at least O(NlogN) time in general. Why would you then sort the array first and then perform binary search instead of just performing linear search?

8.

Consider a farm with **n** cows of two colors, say, white cows and black cows. The cows are arranged in their barn in such a way that all white cows are kept first, followed by all black cows. Each of the cows is assigned a serial number. **You are to find the serial number of the first black cow.**

a. CO1	**Describe** how to convert the given information into an array of integers.	**01**
b. CO1	Given the array of integers from Q(2a), **present** a method with a time complexity **less than O(n)** to solve the mentioned problem in the paragraph. Use pseudocode/ executable code/ step-by-step logical instructions to show your method.	**01**